

Progress Report

Day 7 — Team CRUD & Role-Based Authorization

Task Management MERN Stack Platform

Shafin Nigamana | 25 May 2026

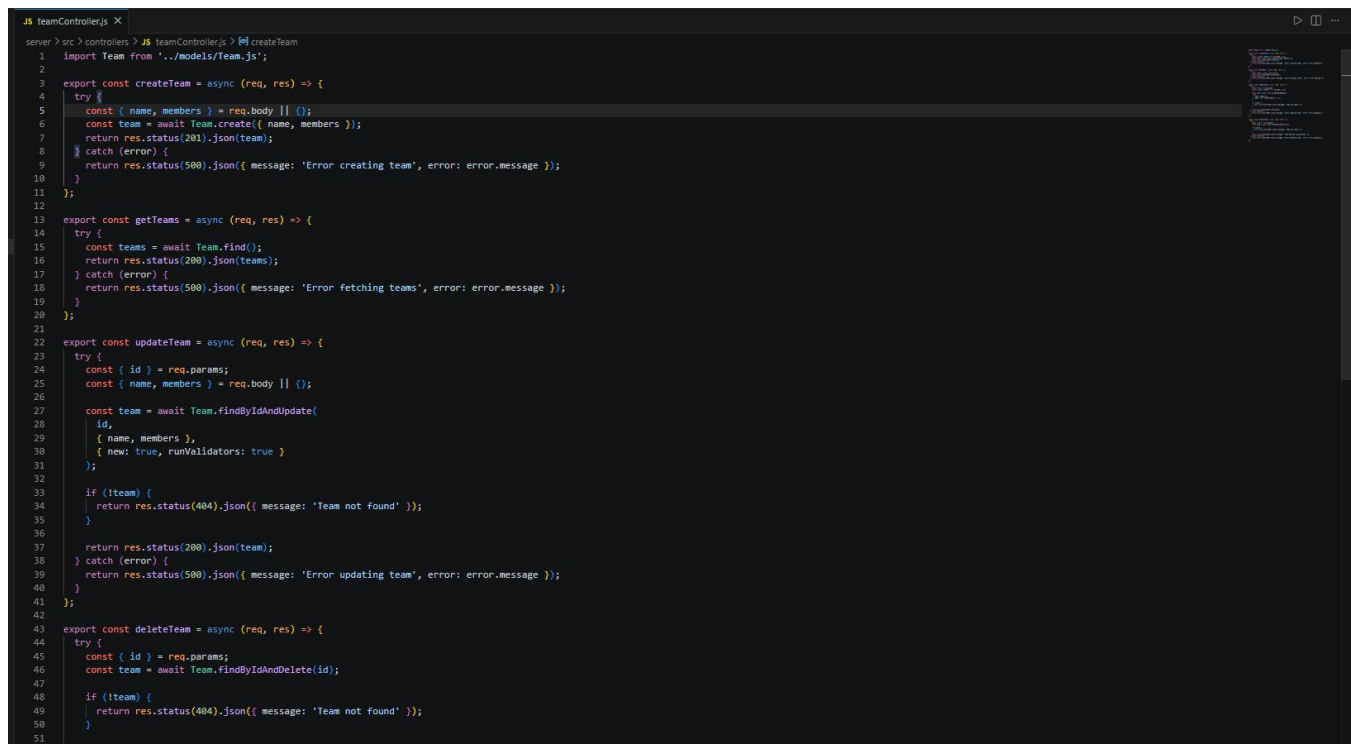
1. Day-7 Objective

The goal for Day 7 was to implement Team CRUD APIs, configure manager-only authorization, and protect backend team routes using JWT authentication middleware.

2. Tasks Completed

Team CRUD Implementation

- Created teamController.js for team management operations
- Integrated Team model with MongoDB collection operations

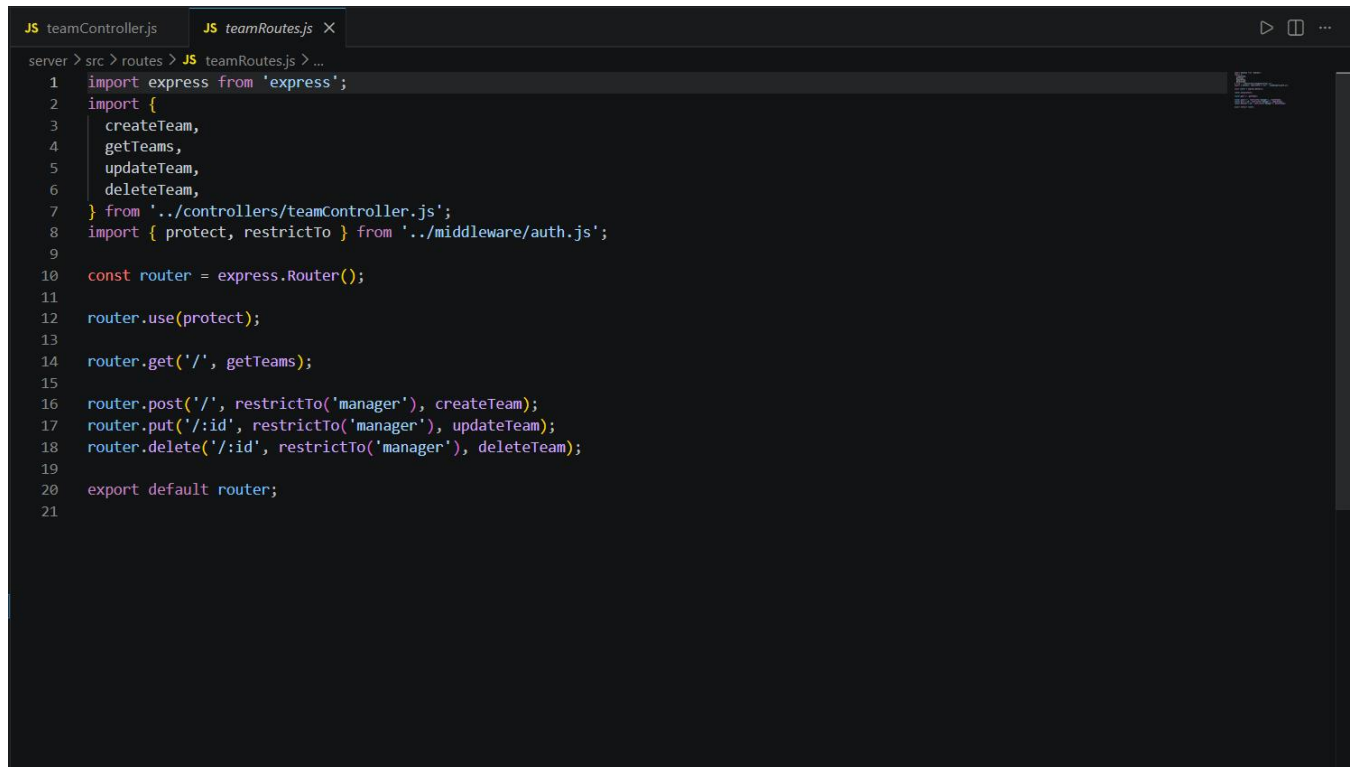


```
server > src > controllers > JS teamController.js > createTeam
1 import Team from '../models/Team.js';
2
3 export const createTeam = async (req, res) => {
4   try {
5     const { name, members } = req.body || {};
6     const team = await Team.create({ name, members });
7     return res.status(201).json(team);
8   } catch (error) {
9     return res.status(500).json({ message: 'Error creating team', error: error.message });
10  }
11 };
12
13 export const getTeams = async (req, res) => {
14   try {
15     const teams = await Team.find();
16     return res.status(200).json(teams);
17   } catch (error) {
18     return res.status(500).json({ message: 'Error fetching teams', error: error.message });
19   }
20 };
21
22 export const updateTeam = async (req, res) => {
23   try {
24     const { id } = req.params;
25     const { name, members } = req.body || {};
26
27     const team = await Team.findByIdAndUpdate(
28       id,
29       { name, members },
30       { new: true, runValidators: true }
31     );
32
33     if (!team) {
34       return res.status(404).json({ message: 'Team not found' });
35     }
36
37     return res.status(200).json(team);
38   } catch (error) {
39     return res.status(500).json({ message: 'Error updating team', error: error.message });
40   }
41 };
42
43 export const deleteTeam = async (req, res) => {
44   try {
45     const { id } = req.params;
46     const team = await Team.findByIdAndDelete(id);
47
48     if (!team) {
49       return res.status(404).json({ message: 'Team not found' });
50     }
51  }
```

Figure 2.1 — Team CRUD controller implementation inside backend architecture.

Team Route Setup

- Created teamRoutes.js inside backend routes structure
- Registered /api/teams routes inside Express application
- Protected team routes using existing JWT authentication middleware



```
server > src > routes > JS teamRoutes.js > ...
1  import express from 'express';
2  import {
3    createTeam,
4    getTeams,
5    updateTeam,
6    deleteTeam,
7  } from '../controllers/teamController.js';
8  import { protect, restrictTo } from '../middleware/auth.js';
9
10 const router = express.Router();
11
12 router.use(protect);
13
14 router.get('/', getTeams);
15
16 router.post('/', restrictTo('manager'), createTeam);
17 router.put('/:id', restrictTo('manager'), updateTeam);
18 router.delete('/:id', restrictTo('manager'), deleteTeam);
19
20 export default router;
21
```

Figure 2.2 — Team routes registered and protected using middleware.

Role-Based Authorization

- Implemented restrictTo('manager') middleware for manager-only endpoints
- Restricted:
 - team creation
 - team update
 - team deletion
- Allowed authenticated users to access team listing routes



Figure 2.3 — Role-based authorization middleware implementation.

Manager Team Creation Verification

- Tested manager role access using Postman
- Verified manager users can successfully create teams

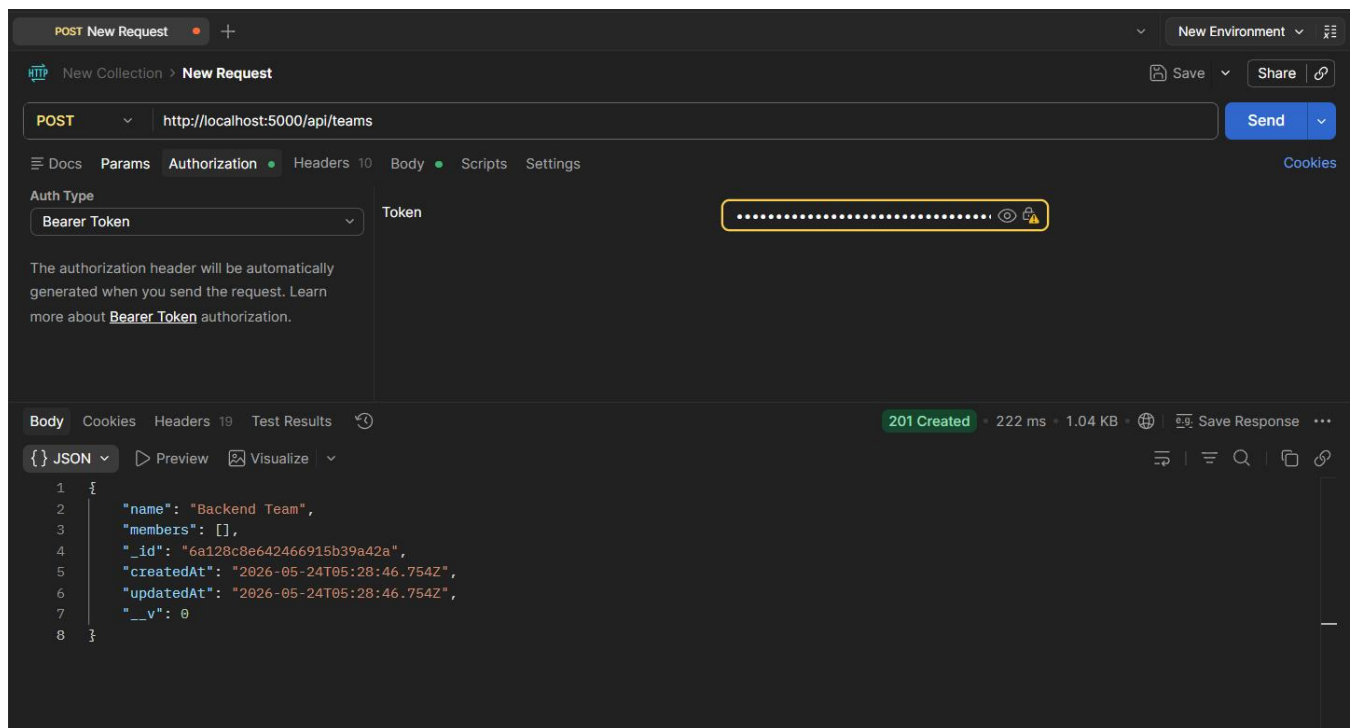


Figure 2.4 — Manager successfully creating a new team.

Member Access Restriction Verification

- Tested member role access against manager-only routes
- Verified backend blocks unauthorized requests using role validation

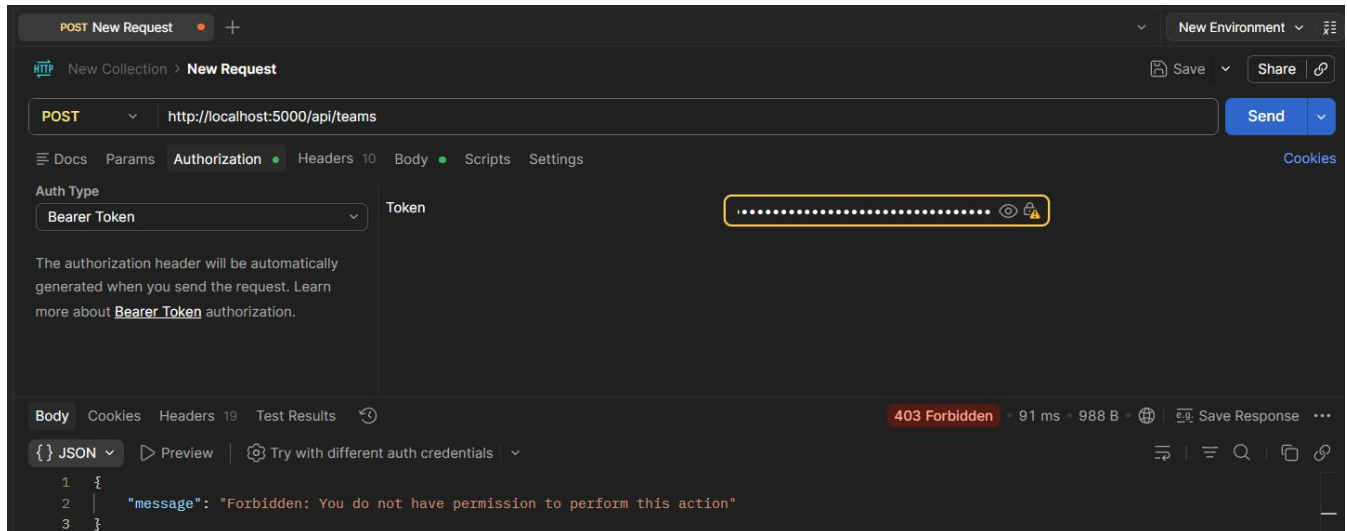


Figure 2.5 — Member role blocked from accessing manager-only endpoint.

Protected Route Verification

- Verified team routes reject unauthenticated requests
- Confirmed JWT authentication middleware protects backend routes correctly

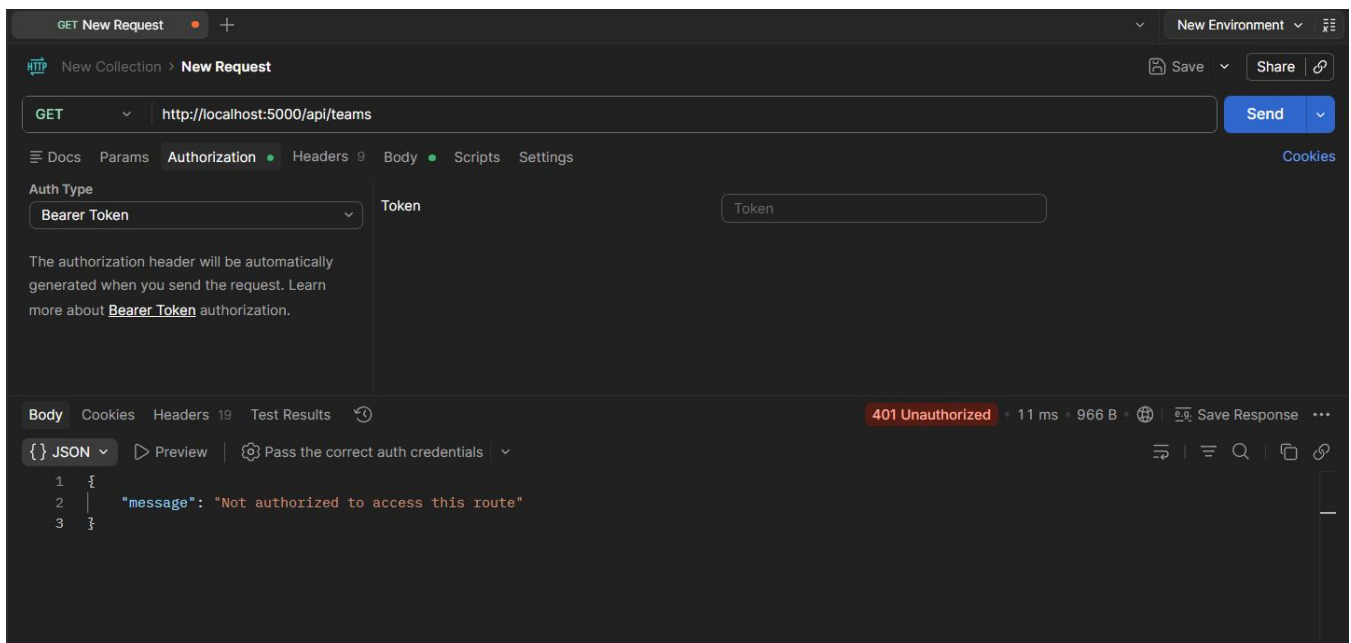


Figure 2.6 — Unauthorized request blocked by authentication middleware.

Team Data Retrieval

- Tested protected GET /api/teams route
- Verified authenticated users can retrieve stored team data

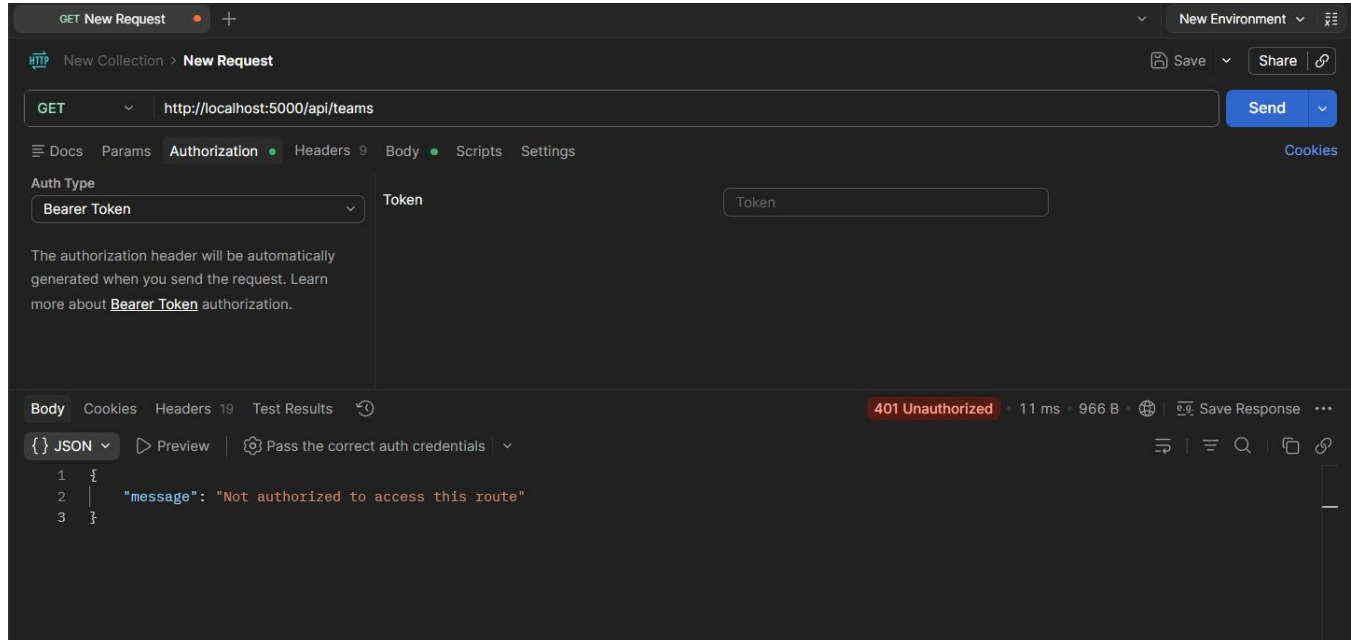


Figure 2.7 — Protected team route returning stored team data.

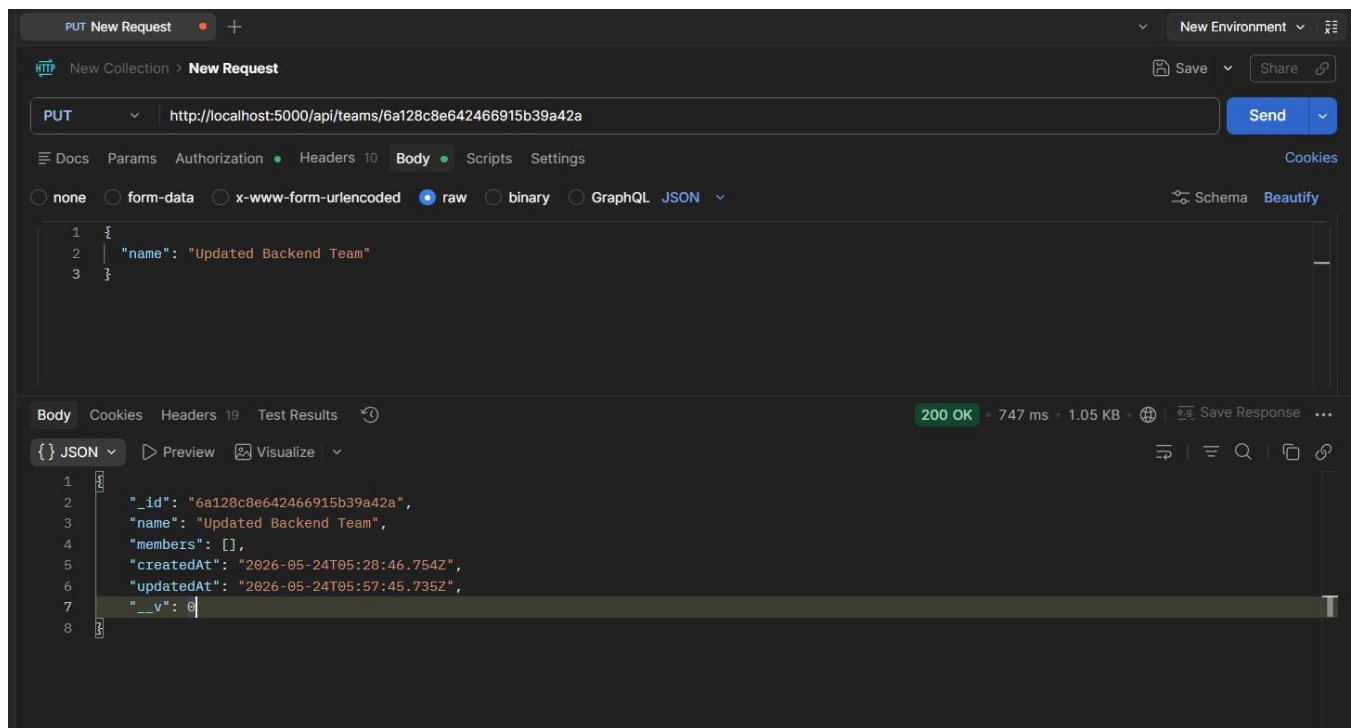


Figure 2.8 — Team update operation verified successfully using manager role authorization.

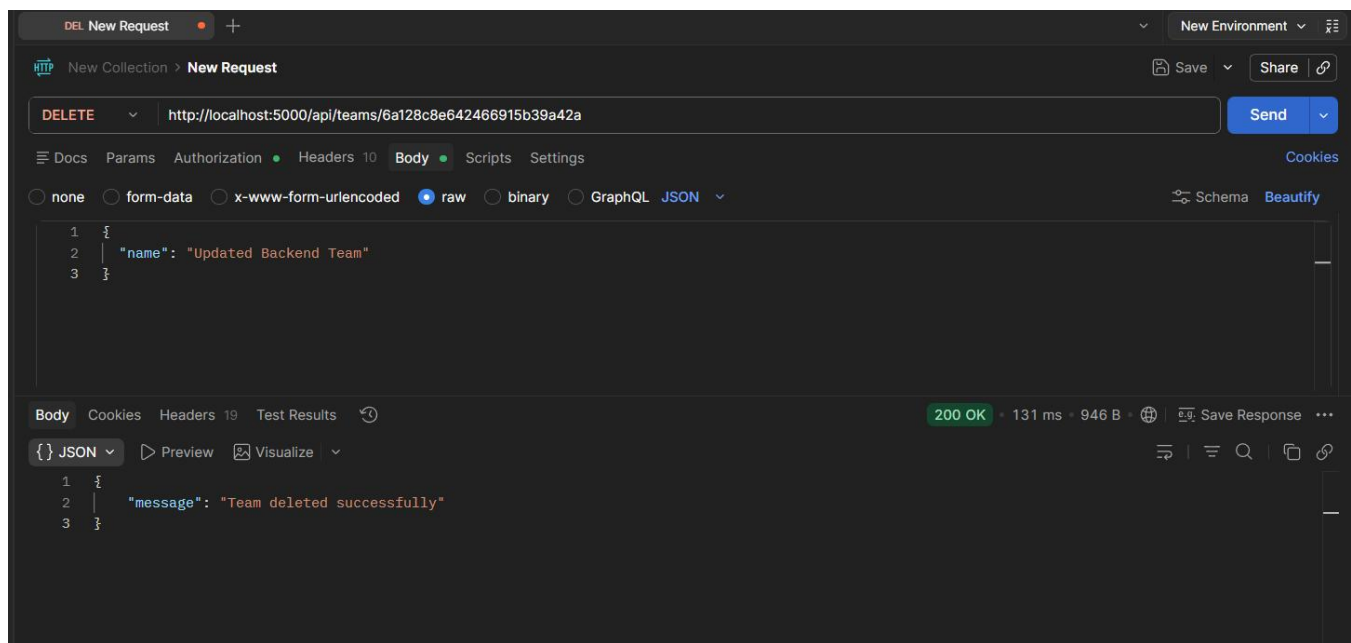


Figure 2.9 — Team delete operation verified successfully through protected backend route.

Repository Update

- Committed Team CRUD and authorization changes using Git workflow
- Pushed latest backend authorization updates to GitHub repository

```
shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/project-setup)
• $ git add .
warning: in the working copy of 'server/src/app.js', LF will be replaced by CRLF the next time Git touches it

shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/project-setup)
• $ git add .

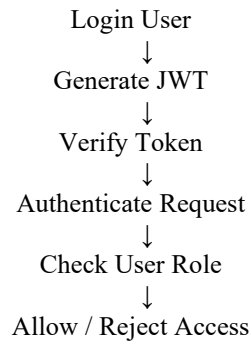
shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/project-setup)
• $ git commit -m "feat: implement team CRUD and role authorization"
[feature/project-setup fc9524e] feat: implement team CRUD and role authorization
4 files changed, 95 insertions(+), 3 deletions(-)
create mode 100644 server/src/controllers/teamController.js
create mode 100644 server/src/routes/teamRoutes.js

shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/project-setup)
• $ git push
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 16 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (11/11), 1.84 KiB | 236.00 KiB/s, done.
Total 11 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To github.com:ShafinNigamana/task-management-mern.git
   63d5cf7..fc9524e  feature/project-setup -> feature/project-setup
```

Figure 2.10 — Day 7 Team CRUD implementation pushed to GitHub.

3. Current Authentication Flow

Current backend authorization flow:



4. Conclusion

Day 7 focused on implementing Team CRUD functionality and role-based authorization. Protected team routes, manager-only access control, and JWT-based authorization were verified successfully using Postman.